

Chapter 5

Step 3: Model Choice, Training, and Application

In the rapidly evolving world of financial trading, leveraging advanced artificial intelligence (AI) methods can significantly enhance predictive accuracy and strategic decision-making.

In AI, a **model** is a mathematical construct trained on a dataset to recognize patterns, make predictions, provide recommendations, or perform other tasks. It represents the learned relationships and structures within the training dataset to be used on the dataset itself, or to make predictions on data the model has not seen.

The two primary types of machine-learning techniques for analyzing data are supervised and unsupervised learning. **Supervised learning** involves training a model on labeled data, where the input features and corresponding output labels are known. It is commonly used for tasks such as classification and regression, where the goal is to predict labels (classification) or continuous values (regression) for new, unseen data. The typical algorithms are linear regression, decision trees, support vector machines, and supervised ranking algorithms. On the other hand, **unsupervised learning** deals with unlabeled data, focusing on discovering hidden patterns or structures within the dataset. Clustering and dimensionality reduction are typical applications of unsupervised learning, utilizing algorithms such as k-means clustering, and principal component analysis (PCA).

Large language models rely on both supervised and unsupervised learning techniques and are thus sometimes called **semi-supervised learning**. First, the model is trained on vast amounts of text data without explicit labels, aiming at predicting the next word in a sentence. Second, the model is usually fine-tuned by training it on a curated set of documents with specific labels.

Reinforcement learning (outlined in a separate chapter) is an individual concept from supervised and unsupervised learning. It is a paradigm where an agent learns to make decisions by taking actions in an environ-

ment to maximize cumulative rewards through trial and error. An example of this model application is the self-driving car.

This chapter introduces various regression techniques and machine-learning algorithms that have become essential tools for modern financial analysts and traders.

We begin with **linear regression**, a fundamental method for modeling the relationship between a dependent variable and one or more independent variables, which is crucial for predicting asset prices based on historical data. **Polynomial regression** extends this by fitting a non-linear relationship, allowing for more complex trend capturing. **Least absolute shrinkage and selection operator (LASSO) regression** introduces regularization to prevent overfitting, effectively handling datasets with numerous predictors. **Ridge regression** similarly adds a penalty term to improve model generalization. **Markov switching dynamic regression** models time series data that switch between different regimes, reflecting market conditions more accurately. **Decision tree regression** provides a non-parametric approach, splitting the data into branches to make predictions based on decision rules. **Support vector machines (SVM) regression with wavelet forecasting** combines the strengths of SVMs and wavelet transformations for precise predictions in noisy environments. A **multiclass random forest model**, which extends the random forest algorithm to handle multiple classes, is effective for classification problems involving more than two categories. **Logistic regression** is pivotal for classification tasks, such as predicting the likelihood of a market event. Moving beyond regression, **hidden Markov models** offer robust frameworks for modeling time series data with hidden states, capturing market regime transitions. **Gaussian Naïve Bayes** applies probabilistic methods to classification, which is particularly useful in high-dimensional datasets. Lastly, **convolutional neural networks (CNN)**, traditionally used in image processing, are adapted for financial time series, capturing intricate patterns and trends.

The **LGBRanker** excels in ordering items based on their relevance or importance. **OPTICS clustering**, known for its ability to identify clusters of varying densities, is ideal for discovering structures in large datasets. The **OpenAI language model**, with its advanced natural language understanding capabilities, is adept at performing complex language tasks. **Amazon Chronos**, designed for time series forecasting, leverages machine learning to provide accurate and scalable predictions. **FinBERT** specializes in financial sentiment analysis and interprets financial texts to gauge market sentiment and inform trading strategies.

Each method is illustrated with a Python example, showcasing its practical applications in financial trading and assessing each model's performance.

Regression

Regression is a statistical method of finding how variables relate to each other. It tries to fit a mathematical model to the dataset to show how a dependent variable depends on independent variables.

In finance, the typical dependent variables are asset prices, and the independent variables may be economic factors or market indicators.

Linear Regression

Description	Linear regression is a fundamental method for modeling the relationship between a dependent variable and one or more independent variables.
Key Use Cases in Finance	Trend analysis, risk management, and forecasting future values of financial metrics, such as stock prices, returns, interest rates, company valuation, or economic indicators.
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn</code>

Linear regression is a fundamental statistical method used to model the relationship between a dependent variable and one or more independent variables.

The key advantages of the method are that it is computationally fast and straightforward to implement and interpret, making it a good starting point for predictive modeling. However, it may not perform well if the relationship is non-linear or if the data has significant outliers.

In finance, it is often used for trend analysis, risk management, and forecasting future values of financial metrics, such as stock prices, returns, interest rates, company valuation, or economic indicators.

Normalization or standardization is not strictly required for linear regression but can improve the performance and interpretability of the model, especially when the features have different scales.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

# Generate Sample Data
# Assuming we have a simple dataset with a linear relationship
np.random.seed(0)
X = np.random.rand(100, 1) * 10 # Feature: Random values between 0 and 10
y = 2.5 * X + np.random.randn(100, 1) * 2 # Target: Linear relationship with noise

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Use the Linear Regression method
model = LinearRegression()
model.fit(X_train, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.scatter(X_test, y_test, color='green', label='Test data')
plt.plot(X_test, model.predict(X_test), color='red', linewidth=2, label='Fitted line')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('Linear Regression')
plt.legend()
plt.show()

# Retrieve model fit statistics
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# A lower MSE and an R^2 score close to 1 indicate a good fit. You can tweak the model by adding m

```

Output (Figure 5.1):

```

Mean Squared Error: 4.173733526278071
R^2 Score: 0.9295849975491168

```

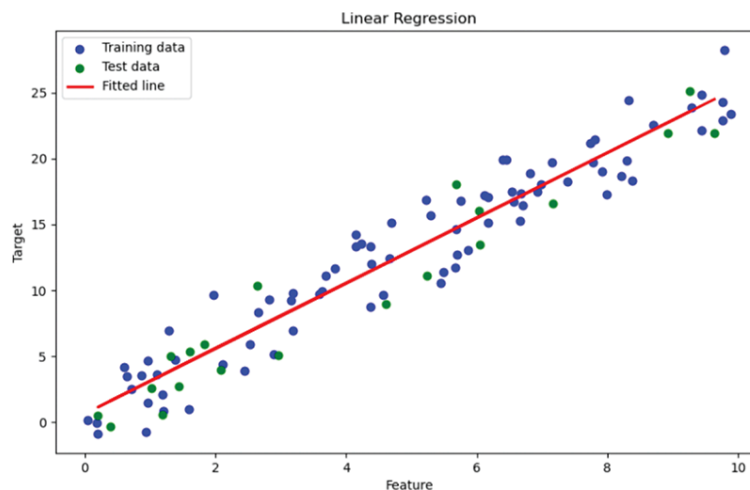


Figure 5.1 Example of linear regression.

Polynomial Regression

Description	Polynomial regression extends linear regression by fitting a non-linear relationship between the independent and dependent variables, allowing for more complex trend capturing in the data.
Key Use Cases in Finance	Modeling the cyclical behavior of financial markets, forecasting interest rates, improving the fit of economic growth models, and analyzing the performance of financial assets.
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<pre>pip install numpy pandas matplotlib scikit-learn</pre>

Polynomial regression is a generalization of linear regression that allows for a curved relationship between the dependent and independent variables. Use polynomial regression when the variables have a non-linear relationship that can be modeled with polynomial terms.

Be careful of overfitting, especially with polynomials of high degree. Regularization techniques are used to prevent overfitting, which works by adding a penalty term to the loss function, which discourages substantial coefficients in the polynomial equation. Two common types of regularization are L1 regularization, which adds the absolute values of the coefficients (leading to lasso regression explained later), and L2 regularization, which adds the squared values of the coefficients (leading to ridge regression explained later). Regularization helps improve the model's performance on unseen data, by avoiding the issue where the model fits the training data too closely, capturing noise instead of the actual underlying patterns.

In finance, polynomial regression helps model more complex relationships in time series data, capturing the cyclical behavior of economic indicators, and improving the fit of models predicting stock prices.

Normalization or standardization can be beneficial, particularly when polynomial terms significantly increase the range of feature values.

Python Example

```
import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 1) * 10 # Feature: Random values between 0 and 10
y = 2.5 * X**2 + np.random.randn(100, 1) * 10 # Target: Quadratic relationship with noise

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Use Polynomial Features and Linear Regression
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.scatter(X_test, y_test, color='green', label='Test data')
X_plot = np.linspace(0, 10, 100).reshape(-1, 1)
y_plot = model.predict(poly.transform(X_plot))
plt.plot(X_plot, y_plot, color='red', linewidth=2, label='Fitted curve')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('Polynomial Regression')
plt.legend()
plt.show()

# Retrieve model fit statistics
y_pred = model.predict(X_test_poly)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# Lower MSE and higher R^2 indicate a good fit. Watch for overfitting by examining the performance

```

With the output ([Figure 5.2](#))

```

Mean Squared Error: 102.85467801527334
R^2 Score: 0.981491014534497

```

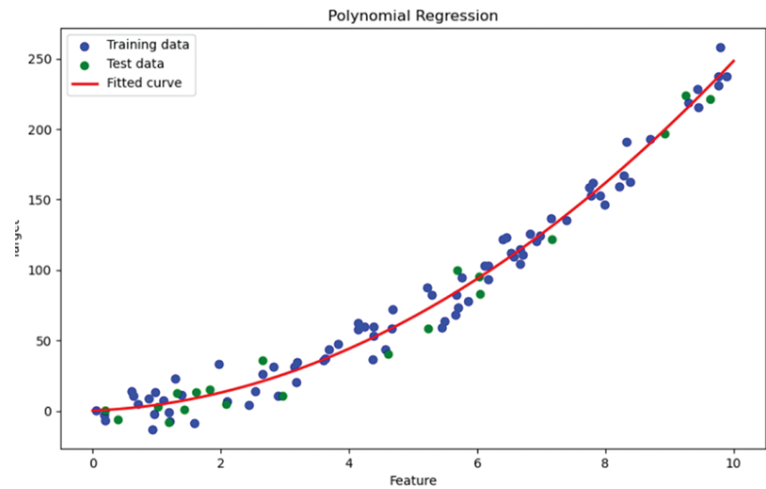


Figure 5.2 Example of polynomial regression.

LASSO Regression

Description	LASSO regression introduces regularization to linear regression to prevent overfitting, effectively handling datasets with numerous predictors.
Key Use Cases in Finance	Feature selection for financial models, predicting credit risk, identifying significant factors affecting stock prices, enhancing portfolio optimization models.
Feature Normalization/Standardization Required	Essential.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn</code>

Least absolute shrinkage and selection operator (LASSO) regression is a type of linear regression that includes a regularization term to shrink the coefficients of less important features to zero. LASSO regression is a good choice when you have many features and think that only some of them are relevant for the prediction of the target variable. It avoids overfitting and enhances model generalization. However, it may not perform well if all features are essential, or if you require an exact model with all features included (Figure 5.3).

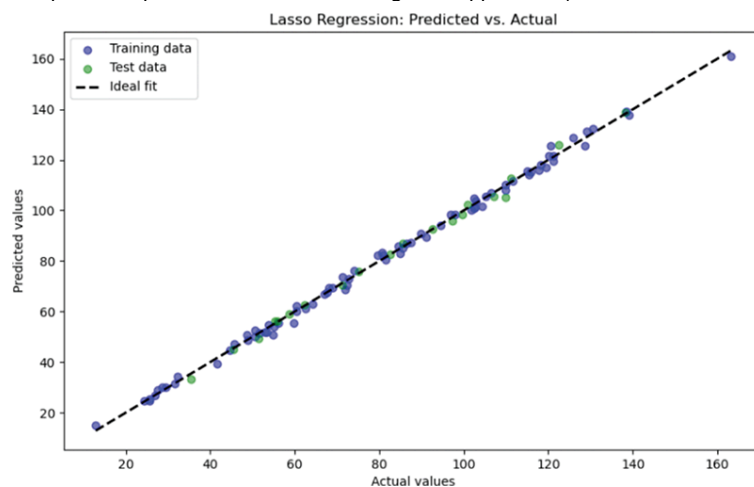


Figure 5.3 Example of LASSO regression.

In LASSO regression, the parameter α controls the regularization strength. The higher the α , the stronger the regularization and the coefficients of less essential features shrink toward zero, effectively performing feature selection.

Formally, the loss function is modified to include a penalty term defined as the sum of the absolute values of the coefficients:

$$Loss = RSS + \alpha \sum_{i=1}^n |w_i|$$

The residual sum of squares (RSS) is defined as the sum of the squared differences between the observed and predicted values, w_i are the model coefficients, and α is the regularization parameter. The LASSO regression aims to minimize the loss function; hence, the higher the α , the lower the coefficients.

In finance, LASSO regression can help identify the most significant predictors of financial outcomes, such as asset returns or credit risk.

Normalization or standardization is essential for LASSO regression to ensure the regularization term is applied uniformly across all features.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 10) # Features: Random values with 10 features
true_coeffs = np.array([22.5, -1.5, 0, 100, 3, 0, 45, 0, 1, 0])
y = X @ true_coeffs + np.random.randn(100) * 2 # Target: Linear combination with noise

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```



```

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use Lasso Regression
model = Lasso(alpha=0.01) # Adjusted alpha for better fit
model.fit(X_train_scaled, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
y_pred_train = model.predict(X_train_scaled)
plt.scatter(y_train, y_pred_train, color='blue', alpha=0.5, label='Training data')
y_pred_test = model.predict(X_test_scaled)
plt.scatter(y_test, y_pred_test, color='green', alpha=0.5, label='Test data')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2, label='Ideal fit')
plt.xlabel('Actual values')
plt.ylabel('Predicted values')
plt.title('Lasso Regression: Predicted vs. Actual')
plt.legend()
plt.show()

# Plot the coefficients
plt.figure(figsize=(10, 6))
plt.plot(model.coef_, marker='o', linestyle='none', label='Lasso coefficients')
plt.xlabel('Feature index')
plt.ylabel('Coefficient value')
plt.title('Lasso Regression Coefficients')
plt.legend()
plt.show()

# Retrieve model fit statistics
mse = mean_squared_error(y_test, y_pred_test)
r2 = r2_score(y_test, y_pred_test)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# Lasso regression coefficients help identify important features. Lower MSE and higher R^2 indicate
# The regularization parameter (alpha) can be tweaked to adjust the amount of shrinkage.

```

With the outputs ([Figures 5.3](#) and [5.4](#))

```

Mean Squared Error: 2.885180122300261
R^2 Score: 0.9961485834952417

```

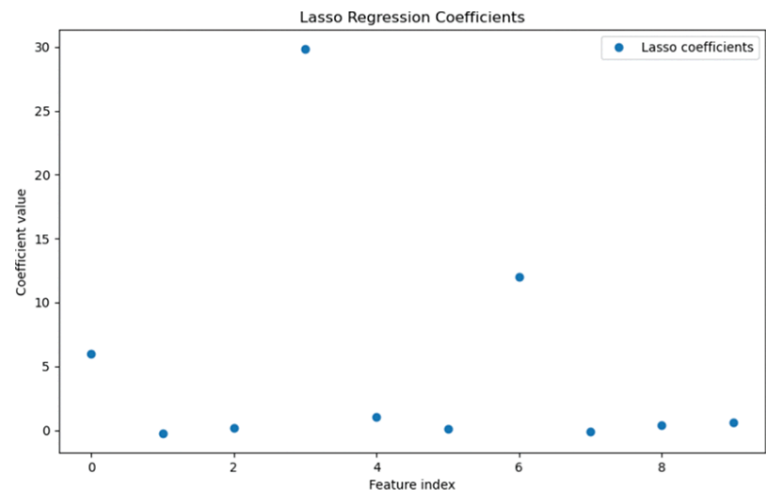


Figure 5.4 Example of LASSO regression coefficients.

Notice that most of the features have close to zero coefficient values.

Ridge Regression

Description	Ridge regression extends linear regression by adding a penalty to the loss function proportional to the sum of the squared coefficients, helping to prevent overfitting by shrinking the coefficients towards zero while maintaining all features in the model.
Key Use Cases in Finance	Forecasting economic indicators, predicting stock returns, reducing model complexity in portfolio optimization, and handling multicollinearity in financial data.
Feature Normalization/Standardization Required	Essential.
Python Libraries Setup	<pre>pip install numpy pandas matplotlib scikit-learn</pre>

Ridge regression is a technique used to analyze multiple regression data that suffers from multicollinearity, a situation when independent variables are highly correlated, making it difficult to isolate the individual effect of each variable on the dependent variable and leading to unreliable coefficient estimates. By adding a degree of bias to the regression estimates, ridge regression can improve the model’s prediction accuracy and interpretability.

In ridge regression, the parameter α (the regularization parameter or penalty term) controls the amount of regularization applied to the model. It balances the trade-off between fitting the training data well and keeping the model coefficients small to prevent overfitting. The formula for the ridge regression loss function is

$$Loss = RSS + \alpha \sum_{i=1}^n w_i^2$$

Where:

- RSS measures the sum of the squared differences between the observed and predicted values.
- α is the regularization parameter. It becomes the ordinary linear regression with no regularization when equal to zero. The higher the α , the more the coefficients are shrunk, at the potential cost of underfitting for its enormous value.
- w_i are the coefficients of the model.

In finance, it is often used for forecasting and reducing model complexity while maintaining predictive performance.

Normalization or standardization is essential to ensure the regularization term is applied uniformly across all features.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 10) # Features: Random values with 10 features
true_coeffs = np.array([20.5, -1.5, 0, 45, 3, 0, 0, 0, 1, 0])
y = X @ true_coeffs + np.random.randn(100) * 2 # Target: Linear combination with noise

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use Ridge Regression
model = Ridge(alpha=1.0)
model.fit(X_train_scaled, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
y_pred_train = model.predict(X_train_scaled)
plt.scatter(y_train, y_pred_train, color='blue', alpha=0.5, label='Training data')
y_pred_test = model.predict(X_test_scaled)
plt.scatter(y_test, y_pred_test, color='green', alpha=0.5, label='Test data')
```

```

plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2, label='Ideal fit')
plt.xlabel('Actual values')
plt.ylabel('Predicted values')
plt.title('Ridge Regression: Predicted vs. Actual')
plt.legend()
plt.show()

# Plot the sample chart with the training and test data and the fitted model
# For high-dimensional data, we focus on coefficients
plt.figure(figsize=(10, 6))
plt.plot(model.coef_, marker='o', linestyle='none', label='Ridge coefficients')
plt.xlabel('Feature index')
plt.ylabel('Coefficient value')
plt.title('Ridge Regression Coefficients')
plt.legend()
plt.show()

# Retrieve model fit statistics
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# Ridge regression coefficients show the impact of regularization. Lower MSE and higher R^2 indicate better model performance.
# The regularization parameter (alpha) can be adjusted to control the amount of shrinkage.

```

With the outputs ([Figures 5.5](#) and [5.6](#))

```

Mean Squared Error: 2.9934174282331862
R^2 Score: 0.982163488280449

```

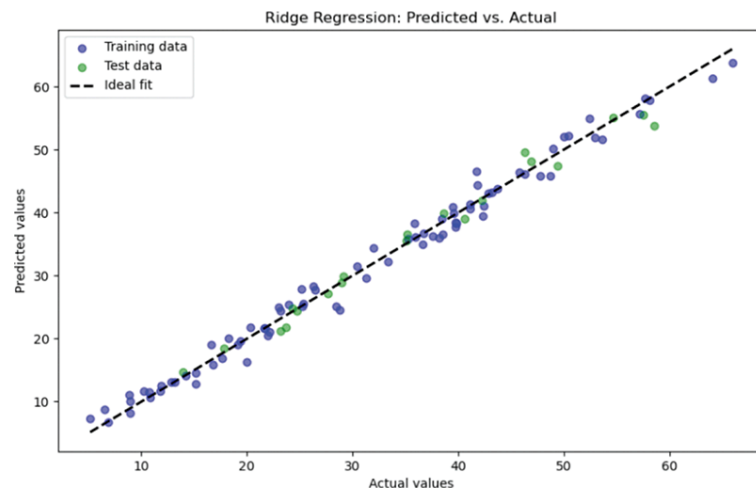


Figure 5.5 Example of ridge regression.

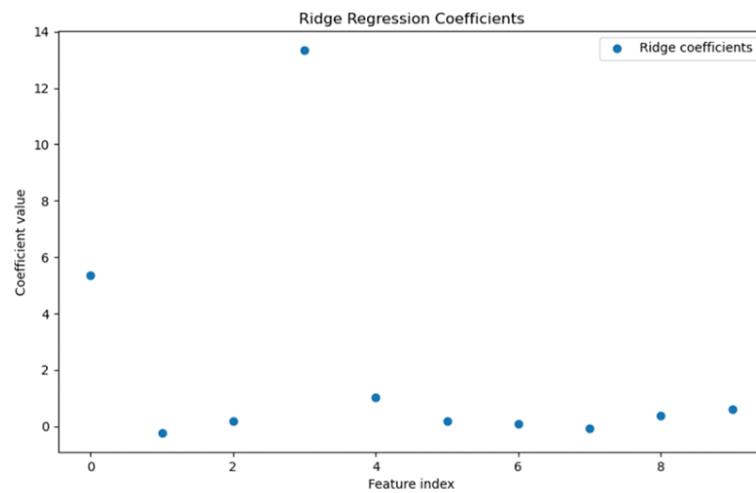


Figure 5.6 Example of ridge regression coefficients.

Notice most of the features' coefficients are essentially zeros.

Markov Switching Dynamic Regression

Description	Markov Switching Dynamic Regression models time series data that switches between different regimes, reflecting market conditions more accurately.
Key Use Cases in Finance	Modeling business cycles and economic growth, detecting bull and bear market regimes, forecasting with structural breaks, enhancing risk management models.
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<pre>pip install numpy pandas matplotlib statsmodels</pre>

Markov (or Markov-switching) regression models are useful for modeling complicated patterns in time series data, where the effect of variables may vary depending on hidden, often unobservable, states. In these models, there are multiple potential states (“regimes”), each with its own sets of regression parameters (e.g., different parameters during the rising and falling markets). A random process drives the switch between the states and depends on only each state (i.e., there is no history dependency).

In these models, there are multiple potential states (“regimes”), each with its own sets of regression parameters (e.g., different parameters during the rising and falling markets). The switch between the states is driven by a random process and depends on only each state (i.e., there is no history dependency).

Markov switching dynamic regression (MSDR) models capture regime changes in time series data. They allow the parameters of a regression model to switch between different states according to a Markov process. They are unsuitable for small datasets or when the regime changes are infrequent or random.

In finance, MSDR models are valuable for modeling economic cycles, detecting market regimes, and improving forecasts by accounting for structural breaks and regime shifts.

Normalization or standardization is recommended to improve convergence and interpretability, especially when dealing with multiple features.

Python Example

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.tsa.regime_switching.markov_regression import MarkovRegression

# Generate Sample Data with Different Volatilities
np.random.seed(0)
n = 200
X = np.linspace(0, 20, n)
regime_1 = 2.5 * X[:n//2] + np.random.randn(n//2) * 5 # Higher volatility in regime 1
regime_2 = -1.5 * X[n//2:] + 30 + np.random.randn(n//2) * 1 # Lower volatility in regime 2
y = np.concatenate([regime_1, regime_2])

# Create a DataFrame
data = pd.DataFrame({'X': X, 'y': y})

# Fit Markov Switching Dynamic Regression
model = MarkovRegression(data['y'], k_regimes=2, trend='c', switching_variance=True)
result = model.fit()

# Plot the observed data and fitted regimes
plt.figure(figsize=(12, 8))

# Plot observed data
plt.plot(data['X'], data['y'], label='Observed', color='blue')

# Extract smoothed probabilities
smoothed_probs = result.smoothed_marginal_probabilities

# Plot the observed data and regime probabilities
for t in range(len(smoothed_probs)):
    if smoothed_probs.iloc[t, 0] > 0.5:
        plt.plot(data['X'].iloc[t], data['y'].iloc[t], 'ro', alpha=0.5)
    else:
        plt.plot(data['X'].iloc[t], data['y'].iloc[t], 'go', alpha=0.5)

# Highlight regime changes
regime_changes = np.argmax(smoothed_probs.values, axis=1)
for i in range(1, len(regime_changes)):
    if regime_changes[i] != regime_changes[i - 1]:
        plt.axvline(x=data['X'].iloc[i], color='gray', linestyle='--', linewidth=1)

plt.xlabel('Feature (X)')
plt.ylabel('Target (y)')
plt.title('Markov Switching Dynamic Regression with Two Regimes')
plt.legend(['Observed', 'Regime 1', 'Regime 2'])
plt.show()

# Plot the regime probabilities
plt.figure(figsize=(12, 6))
plt.plot(data['X'], smoothed_probs[0], label='Regime 1 Probability', color='red')
plt.plot(data['X'], smoothed_probs[1], label='Regime 2 Probability', color='green')
plt.xlabel('Feature (X)')
plt.ylabel('Probability')
plt.title('Smoothed Regime Probabilities')
plt.legend()
plt.show()

```

```
# Retrieve model fit statistics
print(result.summary())

# Interpretation of the results
# The summary provides detailed statistics on the fitted model, including transition probabilities
# These statistics help understand the regime changes and their impact on the dependent variable.
```

With the outputs (Figures 5.7 and 5.8)

Markov Switching Model Results						
=====						
Dep. Variable:	y	No. Observations:	200			
Model:	MarkovRegression	Log Likelihood	-610.192			
Date:	Wed, 31 Jul 2024	AIC	1232.384			
Time:	13:02:59	BIC	1252.174			
Sample:	0	HQIC	1240.393			
	- 200					
Covariance Type:	approx					
	Regime 0 parameters					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	6.2026	0.488	12.714	0.000	5.246	7.159
sigma2	19.6916	2.580	7.631	0.000	14.634	24.749
Regime 1 parameters						
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	17.2055	0.928	18.547	0.000	15.387	19.024
sigma2	35.9522	6.107	5.887	0.000	23.983	47.922
Regime transition parameters						
=====						
	coef	std err	z	P> z	[0.025	0.975]

p[0->0]	0.9937	0.007	150.494	0.000	0.981	1.007
p[1->0]	0.0182	0.015	1.183	0.237	-0.012	0.048
=====						
Warnings:						
[1] Covariance matrix calculated using numerical (complex-step) differentiation.						

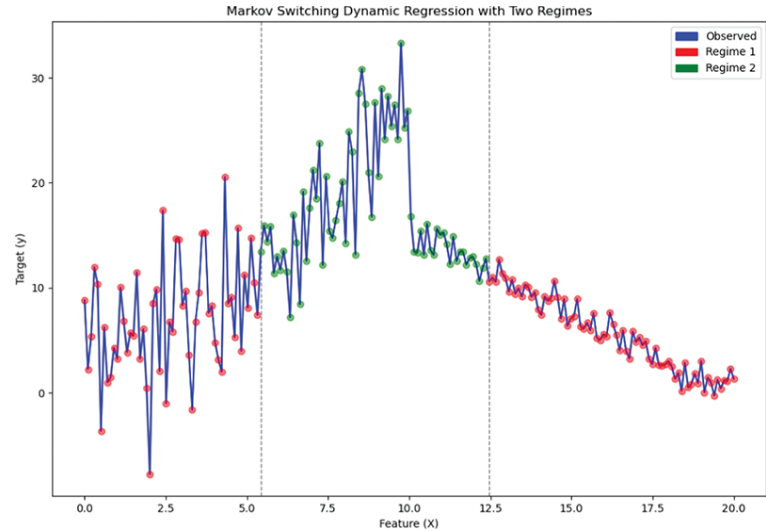


Figure 5.7 Example of Markov switching dynamic regression.

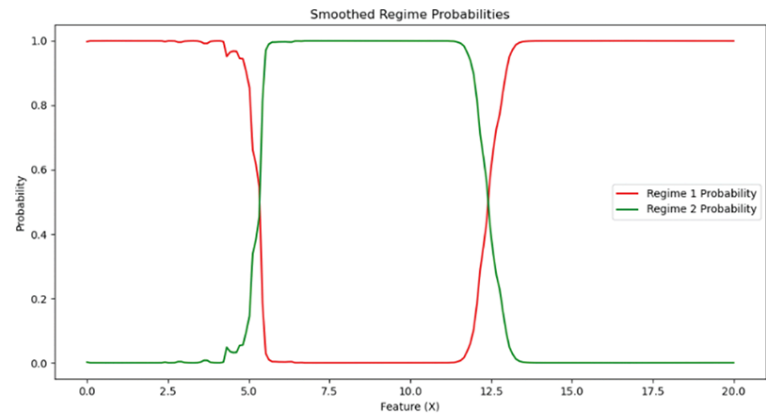


Figure 5.8 Smoothed regime probabilities of the Markov switching dynamic regression.

Decision Tree Regression

Description	Decision tree regression provides a non-parametric approach, splitting the data into branches to make predictions based on decision rules.
Key Use Cases in Finance	Modeling credit risk, forecasting financial distress, identifying key drivers of stock price movements, and predicting loan defaults.
Feature Normalization/Standardization Required	Not required.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn</code>

Decision tree regression is a non-linear predictive modeling technique that partitions the data into subsets based on feature values, resulting in a tree-like structure. Use decision tree regression when you need an interpretable model to capture non-linear relationships without requiring extensive feature engineering. It is prone to overfitting, so pruning techniques or ensemble methods (like random forests) are often used to improve generalization.

In finance, decision tree regression can model complex relationships in data, identify critical determinants of financial outcomes, and provide intuitive visualizations of decision-making processes.

Normalization or standardization is not required for decision tree regression since it is based on splitting rules invariant to feature scaling.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_squared_error, r2_score

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 1) * 10 # Feature: Random values between 0 and 10
y = 2.5 * np.sin(X).ravel() + np.random.randn(100) * 0.5 # Target: Non-linear relationship with noise

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Use Decision Tree Regression
model = DecisionTreeRegressor(max_depth=3)
model.fit(X_train, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.scatter(X_test, y_test, color='green', label='Test data')
X_plot = np.linspace(0, 10, 100).reshape(-1, 1)
y_plot = model.predict(X_plot)
plt.plot(X_plot, y_plot, color='red', linewidth=2, label='Fitted model')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('Decision Tree Regression')
plt.legend()
plt.show()

# Display the generated trees
plt.figure(figsize=(12, 8))
plot_tree(model, filled=True)
plt.title('Decision Tree Structure')
plt.show()

# Retrieve model fit statistics
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# Decision tree regression results include the structure of the tree, showing splits and leaf values
# Lower MSE and higher R^2 indicate a good fit. Pruning can help improve model generalization.
```

With the output (Figures 5.9 and 5.10)

Mean Squared Error: 0.573935429046908
R^2 Score: 0.8041605163184924

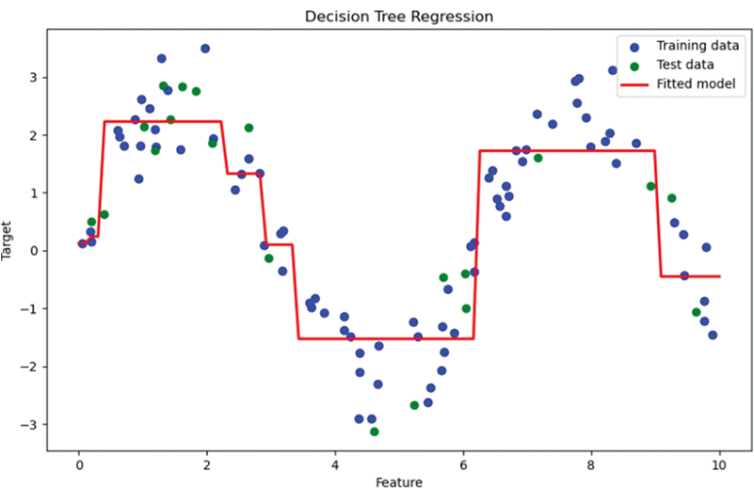


Figure 5.9 Example of decision tree regression.

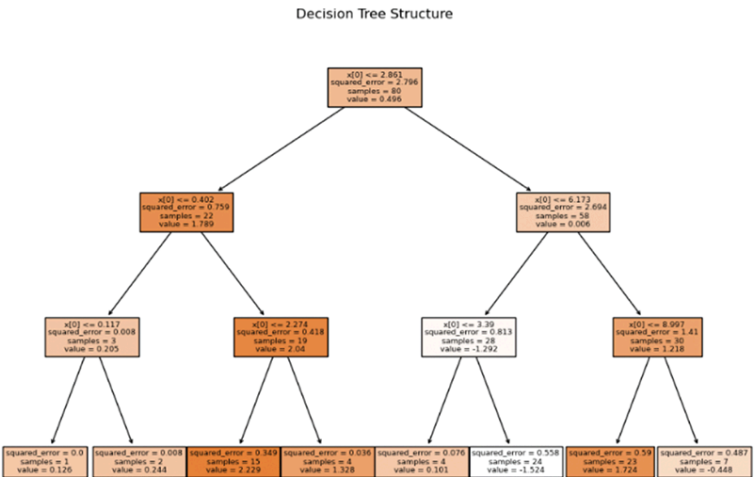


Figure 5.10 Decision tree structure.

Support Vector Machines Regression with Wavelet Forecasting

Description	Support vector machines (SVM) regression with wavelet forecasting combines the strengths of SVMs and wavelet transformations for precise
-------------	--

	predictions in noisy environments.
Key Use Cases in Finance	Stock price forecasting, volatility prediction, high-frequency trading, analyzing cyclical financial data.
Feature Normalization/Standardization Required	Essential.
Python Libraries Setup	<pre>pip install numpy pandas matplotlib scikit-learn pywavelets</pre>

SVM regression, combined with wavelet transforms, is a powerful method for forecasting time series with trends and seasonal patterns.

SVMs are supervised learning models that can perform classification and regression:

- They are effective for classification problems because they can discover the best boundary that separates the data into different groups. For classification problems, SVMs find a hyperplane that optimally divides the data into classes.
- Support vector regression (SVR) aims to find a function that is as smooth as possible and that contains the maximum number of data points within a given margin boundary.

SVM regression captures complex relationships, while wavelet transforms decompose time series data into different frequency components. It may be computationally intensive and less interpretable than simpler models.

It is helpful in finance for financial forecasting; SVR's ability to handle non-linear patterns in financial data makes it ideal for financial and economic forecasting.

Normalization or standardization is essential for SVM to ensure proper convergence and performance, especially when dealing with different scales.

SVM regression has these parameters:

1. **Kernel type:** linear/polynomial/RBF, depending on the nature of data and problem complexity; the default one used by sklearn is RBF.
2. **Regularization parameter (C):** controls the trade-off between a low training error and minimizing the model complexity. A more significant C value might lead to a smaller margin but fewer misclassifications, while a smaller C value creates a more significant margin but more misclassifications.

3. **Epsilon:** the smaller the value, the more precise fit to the training set and the more likely the case of overfitting.
4. **Gamma:** defines how far the influence of a single training example reaches. The default value is auto, but you might try scale or specific numerical values.

Wavelet forecasting is a method in time series analysis that applies wavelets to break down data into different frequency components, revealing patterns hidden in the overall data. This method helps examine non-stationary data where statistical properties vary over time. It separates these components, models them individually, and then uses the outcomes to reassemble and enhance the precision of forecasts. Wavelet forecasting is frequently used in economics, finance, and environmental science.

In finance, wavelet forecasting's ability to effectively handle non-stationary and volatile data enables it to capture short-term and long-term trends and seasonal cycles present in asset prices or economic indicators time series.

In Python, the PyWavelets library performs wavelet decomposition and reconstruction on a signal. It operates in two steps:

Step 1: Wavelet decomposition (`pywt.wavedec`)

Wavelet decomposition is a method of breaking down a signal into its component wavelets at various levels of detail, which allows the simultaneous examination of the signal in both the time and frequency domains.

```
coeffs = pywt.wavedec(y, 'db1', level=2)
```

This function performs a multi-level wavelet decomposition on the signal `y`, where `'db1'` refers to the Daubechies wavelet with one vanishing moment and `level=2` refers to the target output of 2 decomposition levels.

The output is a list of arrays containing the wavelet coefficients.

Step 2: Wavelet reconstruction (`pywt.waverec`)

Reconstructing the original signal from its wavelet coefficients is called wavelet reconstruction and can help with signal denoising, compression, and other signal-processing tasks.

```
y_wavelet = pywt.waverec(coeffs, 'db1')
```

This function reconstructs the signal from the coefficients `coeffs` with the `db1` wavelet type.

The output `y_wavelet` is the reconstructed signal.

Python Example

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
import pywt

# Generate Sample Data
np.random.seed(0)
n = 200
X = np.linspace(0, 20, n).reshape(-1, 1)
y = 2.5 * np.sin(X).ravel() + np.random.randn(n) * 0.5 # Target: Non-linear relationship with noi

# Apply Wavelet Transform
coeffs = pywt.wavedec(y, 'db1', level=2)
y_wavelet = pywt.waverec(coeffs, 'db1')

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y_wavelet, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use SVM Regression
model = SVR(kernel='rbf', C=100, epsilon=0.01)
model.fit(X_train_scaled, y_train)

# Plot the sample chart with the training and test data and the fitted model
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.scatter(X_test, y_test, color='green', label='Test data')
X_plot = np.linspace(0, 20, 200).reshape(-1, 1)
X_plot_scaled = scaler.transform(X_plot)
y_plot = model.predict(X_plot_scaled)
plt.plot(X_plot, y_plot, color='red', linewidth=2, label='Fitted model')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('SVM Regression with Wavelet Forecasting')
plt.legend()
plt.show()

# Retrieve model fit statistics
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Interpretation of the results
# SVM regression with wavelet forecasting results provides insights into the model's predictive ac
# Lower MSE and higher R^2 indicate a good fit. Adjust the wavelet function and SVM parameters to

```

With the outputs ([Figure 5.11](#))

Mean Squared Error: 0.36018129235244006

R² Score: 0.8918512062597711

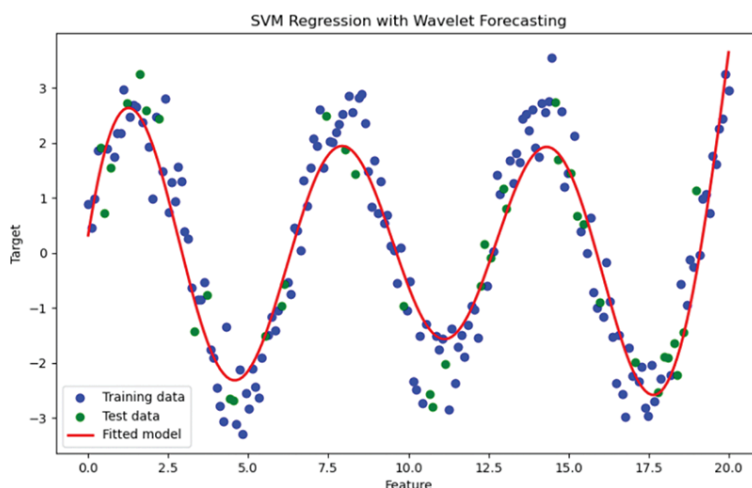


Figure 5.11 Example of SVM regression.

One of the most effective methods of finding the correct values of the SVR method's hyperparameters is by a grid search:

```
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
import pywt

# Generate Sample Data
np.random.seed(0)
n = 200
X = np.linspace(0, 20, n).reshape(-1, 1)
y = 2.5 * np.sin(X).ravel() + np.random.randn(n) * 0.5 # Target: Non-linear relationship with noi

# Apply Wavelet Transform
coeffs = pywt.wavedec(y, 'db1', level=2)
y_wavelet = pywt.waverec(coeffs, 'db1')

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y_wavelet, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use SVM Regression
model = SVR(kernel='rbf')
param_grid = {
    'C': [0.1, 1, 10, 100],
    'epsilon': [0.01, 0.1, 0.5, 1],
    'gamma': ['scale', 'auto', 0.01, 0.1, 1, 10]
}

grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, scoring='r2')
```

```
# Fit the model
grid_search.fit(X_train_scaled, y_train)

# Retrieve the best parameters and best score
best_params = grid_search.best_params_
best_score = grid_search.best_score_

print(f'Best parameters: {best_params}')
print(f'Best R² score: {best_score}')

# Use the best model
best_model = grid_search.best_estimator_

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Test Mean Squared Error: {mse}')
print(f'Test R² Score: {r2}')
```

With the outputs

```
Best parameters: {'C': 10, 'epsilon': 0.5, 'gamma': 10}
Best R² score: 0.9192064453967035
Test Mean Squared Error: 0.2655530910167665
Test R² Score: 0.9202644693735217
```

Classification

Multiclass Random Forest Model

Description	An ensemble learning method that constructs multiple decision trees to classify data into more than two categories.
Key Use Cases in Finance	Fraud detection, customer segmentation, and credit risk assessment.
Feature Normalization/Standardization Required	Not required.
Python Libraries Setup	<code>pip install numpy lightgbm scikit-learn matplotlib seaborn</code>

The multiclass random forest model is an ensemble learning method that builds multiple decision trees to classify data into more than two categories. By aggregating the predictions of several decision trees, as com-

pared to using single decision trees, the model improves accuracy and reduces overfitting.

Key Concepts

- **Ensemble Learning**: ensemble learning is a technique that combines the predictions of multiple machine-learning models to produce a single, more accurate prediction with the motivation that by integrating the strengths of various models, the ensemble can outperform any individual model. Standard ensemble methods include bagging, boosting, and stacking.
- **Decision Tree**: a decision tree is a simple yet powerful predictive model that splits the data into subsets based on the value of input features. Each node in the tree represents a feature, each branch represents a decision rule, and each leaf node represents a class label. Decision trees are easy to interpret and can handle both numerical and categorical data but are prone to overfitting.
- **Random Forest**: a random forest is an ensemble learning method that constructs multiple decision trees during training and merges their predictions to improve accuracy and control overfitting. It uses techniques like bootstrapping (random sampling with replacement) and feature randomness to ensure that the trees are diverse and uncorrelated, resulting in a more robust model that's better generalized to unseen data.
- **Bagging**: bagging, short for bootstrap aggregating, is an ensemble learning technique that trains multiple models on different random subsets of the data and then averages their predictions.

Python Example

We have chosen the LightGBM Python library for its excellent performance in large dataset classification. Its hyperparameters include the following:

- **boosting_type**: the boosting algorithm to use. In this example, we use the 'rf' boosting type (random forest), which is a LightGBM model configured for multiclass classification using multiple trees combined to enhance model performance by minimizing the multiclass logarithmic loss.
- **num_leaves**: the maximum number of leaves in one tree. The larger the number, the more likely the model overfits.
- **learning_rate**: the reduction of the step size used in changing weights. The smaller the number, the more robust the model and the more trees required.
- **feature_fraction**: the fraction of features to consider at each split in tree-building.
- **bagging_fraction**: the fraction of data to be used for each boosting iteration.
- **bagging_freq**: the iteration frequency with which bagging is performed.

We generate sample data using sklearn's `make_classification` method. We use `GridSearchCV` to find the best hyperparameter values.

```

import numpy as np
import lightgbm as lgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Generate sensible sample data using make_classification
n_samples = 1000
n_features = 5
n_classes = 3

X, y = make_classification(n_samples=n_samples, n_features=n_features, n_informative=int(n_feature

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Visualize the distribution of the classes in the dataset
plt.figure(figsize=(10, 6))
sns.countplot(x=y)
plt.title("Distribution of Classes in the Dataset")
plt.xlabel("Class")
plt.ylabel("Frequency")
plt.show()

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'num_leaves': [30, 50, 70],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'feature_fraction': [0.5, 0.6, 0.7, 0.8, 0.9],
    'bagging_fraction': [0.5, 0.6, 0.7, 0.8, 0.9],
    'bagging_freq': [1, 5, 10]
}

# Create a LightGBM model
lgb_estimator = lgb.LGBMClassifier(boosting_type='rf', objective='multiclass', num_class=n_classes

# Perform Grid Search
grid_search = GridSearchCV(estimator=lgb_estimator, param_grid=param_grid, scoring='accuracy', cv=
grid_search.fit(X_train, y_train)

# Get the best parameters and accuracy from the grid search
best_params = grid_search.best_params_
best_accuracy = grid_search.best_score_
print("Best Parameters:", best_params)
print("Best Accuracy:", best_accuracy)

# Train the model with the best parameters
model = lgb.LGBMClassifier(boosting_type='rf', objective='multiclass', num_class=n_classes, **best
model.fit(X_train, y_train, eval_set=[(X_test, y_test)])

# Predict and evaluate the model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy after tuning:", accuracy)
print("Classification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)

```

```
plt.figure(figsize=(10, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=range(n_classes), yticklab
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("True Class")
plt.show()

# Plot feature importance
plt.figure(figsize=(10, 6))
lgb.plot_importance(model, max_num_features=10, importance_type="gain")
plt.title("Feature Importance")
plt.show()
```

With outputs ([Figures 5.12](#), [5.13](#), and [5.14](#))

```
Accuracy after tuning: 0.875
Classification Report:

```

	precision	recall	f1-score	support
0	0.88	0.89	0.88	56
1	0.90	0.81	0.85	74
2	0.86	0.93	0.89	70
accuracy			0.88	200
macro avg	0.88	0.88	0.88	200
weighted avg	0.88	0.88	0.87	200

Notice the classification classes are approximately balanced.

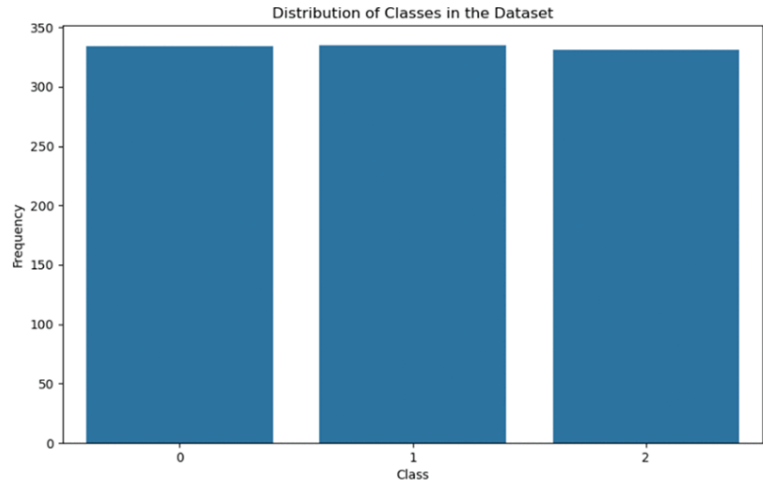


Figure 5.12 Example of class distribution in a dataset.

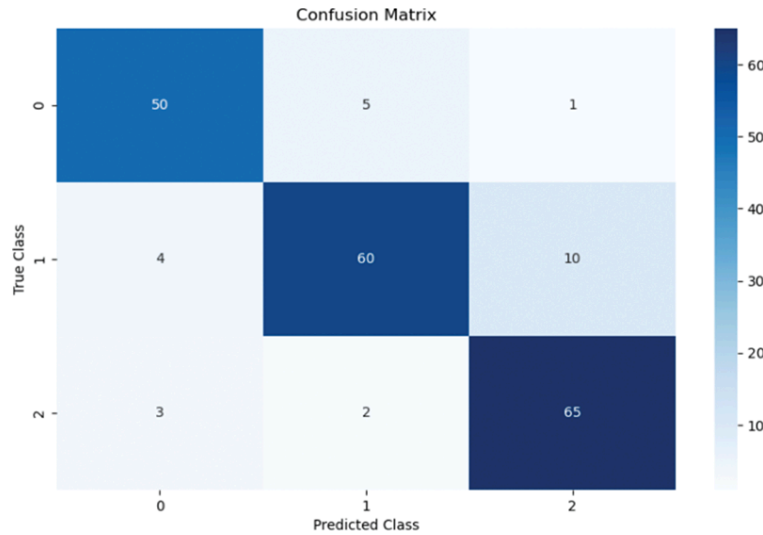


Figure 5.13 Example of confusion matrix.

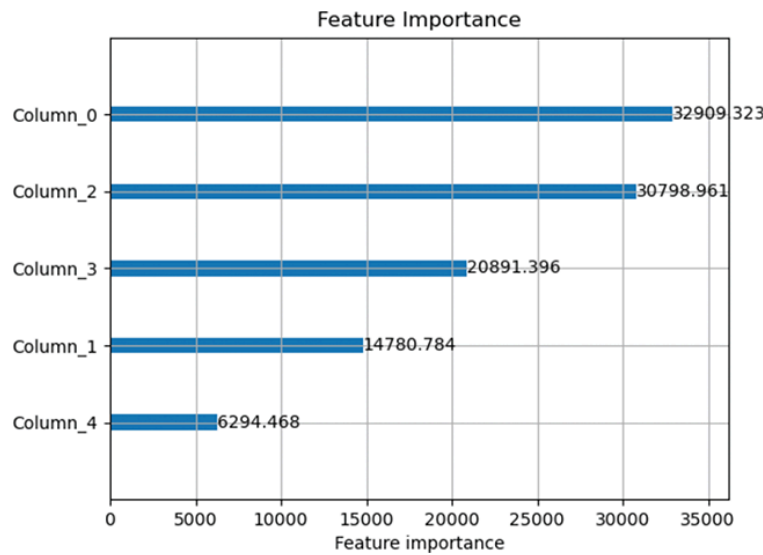


Figure 5.14 Example of feature importance.

Logistic Regression

Description	Logistic Regression is pivotal for classification tasks, such as
-------------	--

	predicting the likelihood of a market event.
Key Use Cases in Finance	Predicting credit default, fraud detection, loan approval decisions, buy/sell recommendations.
Feature Normalization/Standardization Required	Recommended.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn</code>

Logistic regression is widely used in finance for binary classification problems (e.g., default/no default, buy/sell). It provides probabilities and can be extended to multiclass classification. However, it assumes a linear relationship between the predictors and the log odds of the outcome, which may not hold in all cases.

In finance, logistic regression can predict whether a loan applicant will default, whether a transaction is fraudulent, or whether a customer will buy a financial product. By estimating the odds of different outcomes, logistic regression helps financial institutions make informed decisions, manage risk, and develop targeted marketing strategies.

Normalization or standardization is recommended, especially when predictors have different scales, to ensure the model converges appropriately and to improve interpretability.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 10) # Features: Random values with 10 features
true_coeffs = np.array([2.5, -1.5, 0, 0, 3, 0, 0, 0, 1, 0])
logits = X @ true_coeffs + np.random.randn(100) * 2
y = (logits > np.median(logits)).astype(int) # Target: Binary outcome based on logits

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```

# Use Logistic Regression
model = LogisticRegression()
model.fit(X_train_scaled, y_train)

# Plot the sample chart with the training and test data and the fitted model
# For binary classification, we plot the ROC curve
y_pred_prob = model.predict_proba(X_test_scaled)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

# Retrieve model fit statistics
print("Classification Report")
print(classification_report(y_test, model.predict(X_test_scaled)))
print("Confusion Matrix")
print(confusion_matrix(y_test, model.predict(X_test_scaled)))

# Interpretation of the results
# The classification report and confusion matrix provide detailed metrics on the model's performan
# A higher AUC indicates better model performance. Adjust regularization parameters to control ove

```

With the outputs ([Figure 5.15](#))

```

Classification Report
              precision    recall  f1-score   support

         0       0.55      0.75      0.63         8
         1       0.78      0.58      0.67        12

   accuracy              0.65         20
  macro avg       0.66      0.67      0.65         20
 weighted avg       0.68      0.65      0.65         20

Confusion Matrix
[[6 2]
 [5 7]]

```

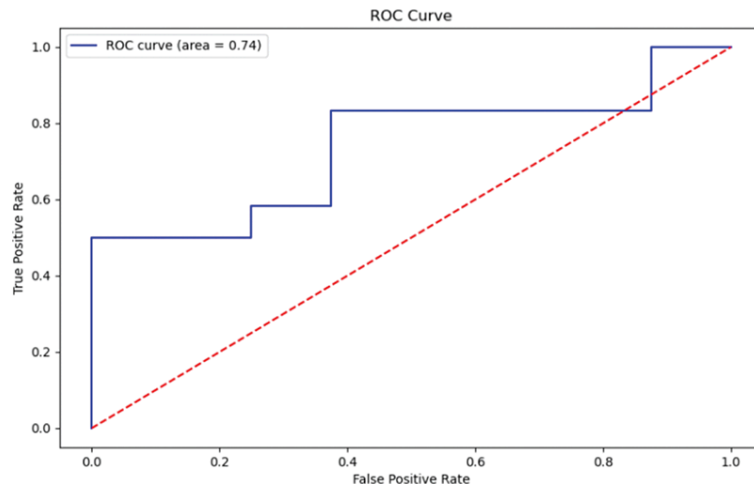


Figure 5.15 Example of ROC curve.

For classification models, a **confusion matrix** is a table used to describe the performance of a model by comparing the predicted labels with the actual labels. It is a 2×2 matrix for binary classification:

- **True Positive (TP)**: the number of instances correctly predicted as the positive class.
- **True Negative (TN)**: the number of instances correctly predicted as the negative class.
- **False Positive (FP)**: the number of instances incorrectly predicted as the positive class (Type I error).
- **False Negative (FN)**: the number of instances incorrectly predicted as the negative class (Type II error).

The confusion matrix looks like this:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

The following key metrics are derived from the confusion matrix:

1. **Accuracy**: the ratio of correctly predicted instances (both positive and negative) to the total cases.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. **Precision (Positive Predictive Value)**: the ratio of correctly predicted positive instances to the total predicted positives.

$$Precision = \frac{TP}{TP + FP}$$

3. **Recall (Sensitivity or True Positive Rate)**: the ratio of correctly predicted positive instances to the total actual positives.

$$Recall = \frac{TP}{TP + FN}$$

4. **F1 Score:** the harmonic mean of precision and recall, providing a balance between the two.

$$F1\ Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

The **receiver operating characteristics (ROC) curve** plots the actual positive rate (sensitivity) against the false positive rate (1 – specificity) at various threshold levels, allowing you to assess the model’s ability to distinguish between positive and negative classes; a higher area under the curve (AUC) value indicates better overall model performance, with values closer to 1 representing excellent performance.

Hidden Markov Models

Description	Hidden Markov models offer robust frameworks for modeling time series data with hidden states, capturing market regime transitions.
Key Use Cases in Finance	Modeling market regimes, pattern recognition in time series data, predicting economic cycles, and improving trading strategies.
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<code>pip install numpy pandas matplotlib hmmlearn</code>

Hidden Markov models (HMMs) are statistical models that represent systems with hidden states and provide probabilistic state predictions. However, they can be complex to train and require careful parameter estimation.

In finance, HMMs are used to model market regimes, detect patterns in time series data, and improve forecasting by capturing the probabilistic transitions between states.

Normalization or standardization can improve model convergence and interpretability, especially when dealing with multiple features.

HMMs take the number of states (“regimes”) as input. To find the correct number of states, start with two and increase depending on the result.

Python Example

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from hmmlearn import hmm

# Generate Sample Data
np.random.seed(0)
n = 200
X = np.linspace(0, 20, n)
state_1 = 2.5 * np.sin(X[:n//2]) + np.random.randn(n//2) * 0.5
state_2 = -1.5 * np.sin(X[n//2:]) + 2 + np.random.randn(n//2) * 0.5
y = np.concatenate([state_1, state_2])

# Fit Hidden Markov Model
model = hmm.GaussianHMM(n_components=2, covariance_type="full", n_iter=100)
model.fit(y.reshape(-1, 1))

# Predict hidden states
hidden_states = model.predict(y.reshape(-1, 1))

# Plot the sample chart with the hidden states
plt.figure(figsize=(10, 6))
plt.plot(X, y, label='Observed', color='blue')
plt.plot(X[hidden_states == 0], y[hidden_states == 0], 'ro', label='State 1')
plt.plot(X[hidden_states == 1], y[hidden_states == 1], 'go', label='State 2')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('Hidden Markov Model')
plt.legend()
plt.show()

# Retrieve model fit statistics
print('Transition Matrix')
print(model.transmat_)
print('Means and Variances of Each State')
print(model.means_)
print(model.covars_)

# Interpretation of the results
# The transition matrix shows the probabilities of moving from one state to another.
# The means and variances provide insights into the characteristics of each hidden state.

```

With the outputs ([Figure 5.16](#))

```

Transition Matrix
[[0.98722454 0.01277546]
 [0.05087947 0.94912053]]
Means and Variances of Each State
[[ 2.05014575]
 [-1.43768284]]
[[[1.13209908]]
 [[1.05456691]]]

```

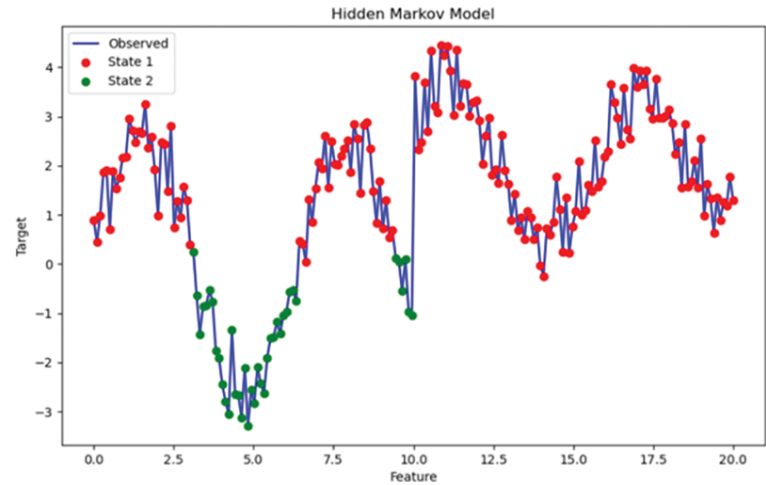


Figure 5.16 Example of hidden Markov model.

Gaussian Naive Bayes

Description	Gaussian Naive Bayes applies probabilistic methods to classification, which is particularly useful in high-dimensional datasets.
Key Use Cases in Finance	Classifying credit risk, predicting market trends, anomaly detection in trading data, fraud detection.
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn</code>

The Gaussian Naive Bayes algorithm is a classification method that detects spam, diagnoses diseases, or is used for fraud detection. The technique uses Bayes’ Theorem, assuming features have a normal (Gaussian) shape. It learns the mean and variance of features from the training data and uses them to estimate the probabilities for each class. Then, it chooses the class with the highest probability for the new data.

Gaussian Naive Bayes is effective for classification tasks with normally distributed features. It is simple to implement and interpret but may underperform if the feature distribution deviates significantly from normality or the feature independence assumption does not hold.

It is helpful in finance for classifying financial risks, predicting market movements, and detecting anomalies in trading patterns.

Normalization or standardization can improve model performance, especially when features have different scales or are not normally distributed.

Python Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

# Generate Sample Data
np.random.seed(0)
X = np.random.rand(100, 10) # Features: Random values with 10 features
true_coeffs = np.array([2.5, -1.5, 0, 0, 3, 0, 0, 0, 1, 0])
logits = X @ true_coeffs + np.random.randn(100) * 2
y = (logits > np.median(logits)).astype(int) # Target: Binary outcome based on logits

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train_scaled, y_train)

# Plot the sample chart with the training and test data and the fitted model
# For binary classification, we plot the ROC curve
y_pred_prob = model.predict_proba(X_test_scaled)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

# Retrieve model fit statistics
print("Classification Report:")
print(classification_report(y_test, model.predict(X_test_scaled)))
print("Confusion Matrix:")
print(confusion_matrix(y_test, model.predict(X_test_scaled)))

# Interpretation of the results
# The classification report and confusion matrix provide detailed metrics on the model's performan
```

With outputs ([Figure 5.17](#))

Classification Report:					
	precision	recall	f1-score	support	
0	0.64	0.88	0.74	8	
1	0.89	0.67	0.76	12	
accuracy			0.75	20	
macro avg	0.76	0.77	0.75	20	
weighted avg	0.79	0.75	0.75	20	
Confusion Matrix:					
[[7 1]					
[4 8]]					

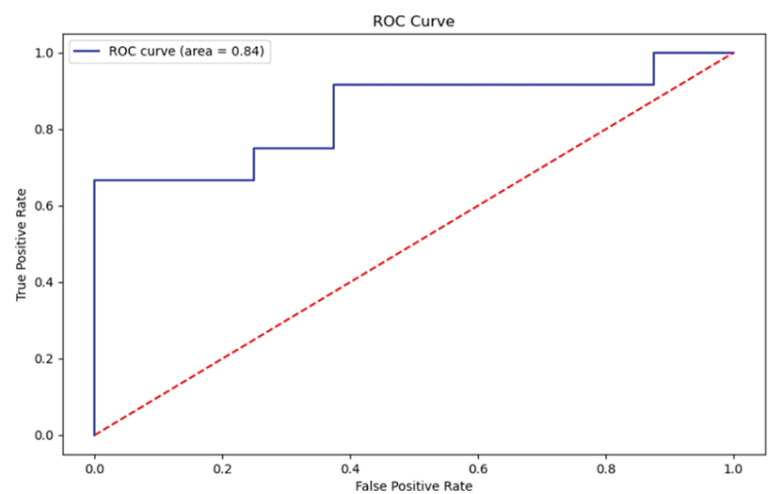


Figure 5.17 ROC curve of Gaussian Naive Bayes.

Convolutional Neural Networks

Description	Convolutional neural networks (CNN), traditionally used in image processing, are adapted here for financial time series, capturing intricate patterns and trends.
Key Use Cases in Finance	Stock price prediction, anomaly detection in financial data, high-frequency trading, sentiment analysis from financial news.
Feature Normalization/Standardization Required	Essential.
Python Libraries Setup	<code>pip install numpy pandas matplotlib scikit-learn tensorflow</code>

Machine-learning models trying to imitate how the human brain works are called neural networks. These models have interconnected layers of nodes or “neurons.” Using a mathematical function, they transform the input data to produce output. Training involves learning from data by changing their connections (or weights) to predict the correct output.

CNNs are deep learning models initially designed for image processing but have been adapted for various applications, including time series forecasting in finance. CNNs can capture complex patterns and dependencies in data that simpler models cannot. CNNs require extensive computational resources and tuning but offer high predictive power and flexibility. They may be unsuitable for small datasets or where model interpretability is crucial.

In finance, CNNs help predict stock prices, detect anomalies, and analyze financial time series data.

Normalization or standardization is essential to ensure faster convergence and better performance.

To build an effective neural network model, the first step is to choose the appropriate model architecture and determine the number of layers. Key model architectures include feedforward neural networks for simple pattern recognition, CNN for recognizing complex patterns or features, recurrent neural networks (RNN) for time series data and understanding context, and transformer models for natural language processing.

Once the architecture is selected, the next step is to architect the model. This step involves deciding on the size of the input layer, determining the number and connections between the hidden layers, and defining the size of the output layer.

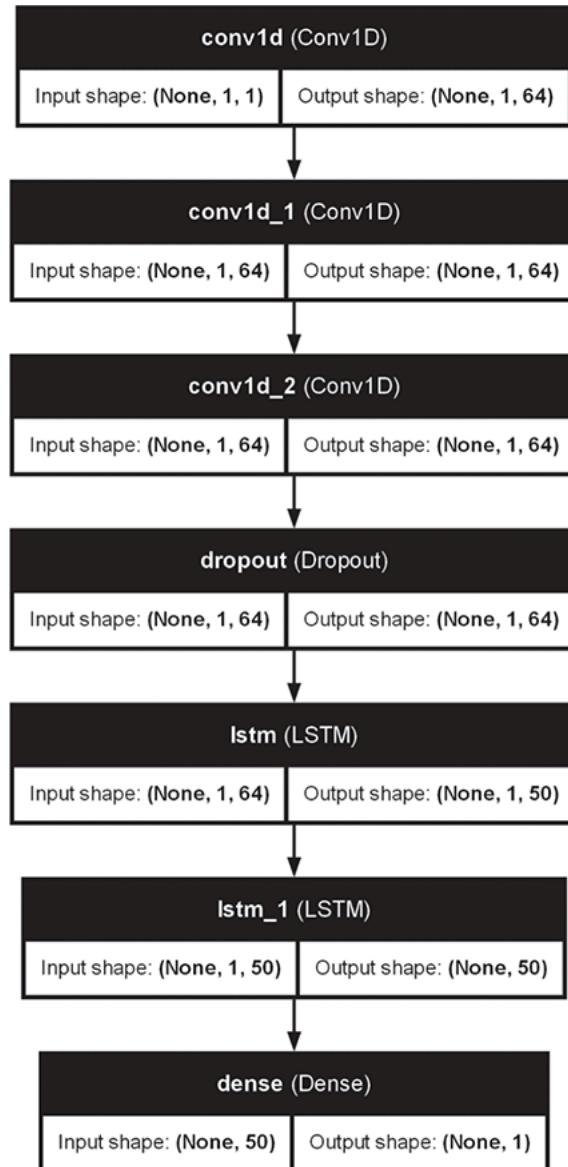
Choosing the model’s hyperparameters and methods to avoid overfitting is crucial for optimal performance. Key hyperparameters include the loss function, which measures the quality of the model’s performance, and the optimizer, which updates model weights based on the loss function values. Overfitting avoidance methods, such as regularization, dropout, early stopping, and data augmentation, are essential to ensure the model generalizes well to unseen data.

Python Example

In the following example, we construct this neural network:

- The [Keras/TensorFlow](#) framework has been chosen due to its simplicity and ease of use, as well as the high-speed performance provided by TensorFlow.
- We use
 - [convolutional layers](#) for their ability to capture effectively complex patterns;
 - [long short-term memory \(LSTM\)](#) layers, which are designed to learn from sequential data by maintaining long-term dependencies; and

- a [dropout layer](#), a regularization technique preventing overfitting by randomly setting a fraction of the input units to zero during training.
- We train the neural network with the [Adam \(adaptive moment estimation\) optimization algorithm](#) because it has low memory requirements, is computationally efficient, and works well with large datasets.



```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, Dense, Flatten, Dropout, LSTM
import tensorflow as tf

# Enable logging
tf.get_logger().setLevel('INFO')

```

```

# Generate Sample Data
np.random.seed(0)
n = 1000
X = np.linspace(0, 1000, n)
y = np.sin(X/100) # Target: Long sinusoidal wave

# Plot the sine wave
plt.figure(figsize=(10, 6))
plt.plot(X, y, label='sin(x)')
plt.xlabel('X')
plt.ylabel('sin(X)')
plt.title('Sine Wave')
plt.legend()
plt.grid(True)
plt.show()

# Reshape data for CNN
X = X.reshape(-1, 1)
y = y.reshape(-1, 1)

# Split the data into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Reshape for CNN input
X_train_scaled = X_train_scaled.reshape((X_train_scaled.shape[0], 1, 1))
X_test_scaled = X_test_scaled.reshape((X_test_scaled.shape[0], 1, 1))

# Use CNN-LSTM for Regression
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=1, activation='relu', input_shape=(1, 1)))
model.add(Conv1D(filters=64, kernel_size=1, activation='relu'))
model.add(Conv1D(filters=64, kernel_size=1, activation='relu'))
model.add(Dropout(0.5))
model.add(LSTM(50, return_sequences=True))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Fit the model with logging
history = model.fit(X_train_scaled, y_train, epochs=300, batch_size=32, verbose=1, validation_spli

# Plot the sample chart with the training and test data and the fitted model
y_pred = model.predict(X_test_scaled)

plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='green', label='Test data')
plt.scatter(X_test, y_pred, color='red', label='Fitted model')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.title('CNN-LSTM Neural Network Regression')
plt.legend()
plt.show()

# Retrieve model fit statistics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

```

```

print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')

# Plot training & validation loss values
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Train loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.title('Model loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.show()

# Interpretation of the results
# CNN-LSTM regression results provide insights into the model's ability to capture complex pattern
# Lower MSE and higher R^2 indicate a good fit. Adjust the network architecture and parameters to

```

With the outputs ([Figures 5.18](#) and [5.19](#))

```

Epoch 1/300
20/20 _____ 2s 13ms/step - loss: 0.4776 - val_loss: 0.3855
...
Epoch 300/300
20/20 _____ 0s 3ms/step - loss: 0.0018 - val_loss: 0.0759
7/7 _____ 0s 27ms/step
Mean Squared Error: 0.08142697718792646
R^2 Score: 0.8100884046552388

```

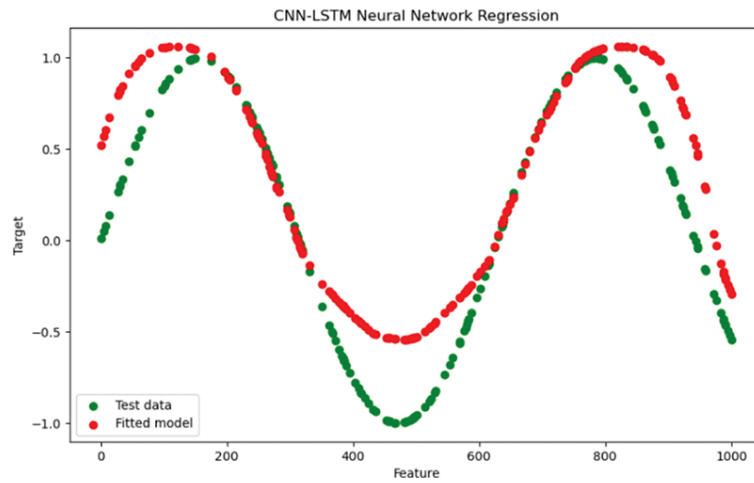



Figure 5.18 Example of neural network regression.

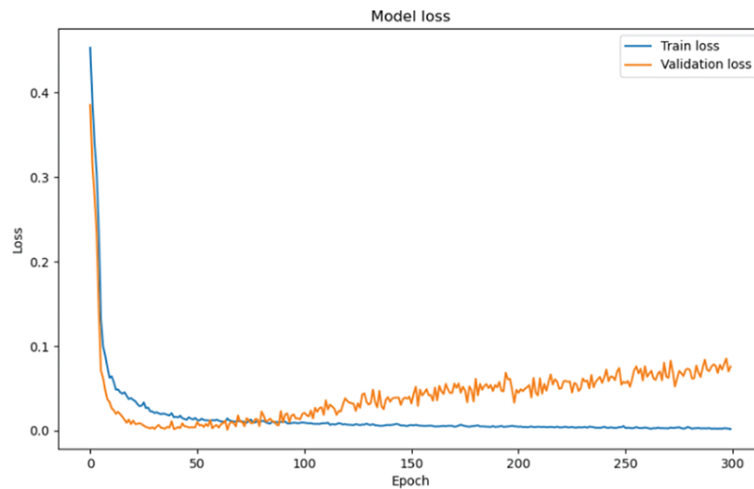


Figure 5.19 Example of model loss.

The model loss chart indicates the following:

- The training loss is consistently decreasing, indicating the model is effectively learning from the training data.
- However, the validation loss is increasing, meaning the model is overfitting, where the model is learning noise and details from the training data, and does not effectively generalize to the new data.

Ranking

LGBRanker Ranking

Description	LGBRanker is a ranking algorithm from the LightGBM library designed to optimize ranking-specific objectives.
Key Use Cases in Finance	Credit risk modeling (ranking customers by default risk), recommendation systems (prioritizing financial products for customers), customer ranking (ranking clients by potential value or risk), and portfolio optimization (ranking assets for investment decisions).
Feature Normalization/Standardization Required	Not required but recommended.
Python Libraries Setup	<code>pip install numpy lightgbm matplotlib scikit-learn</code>

LGBRanker is an algorithm from the LightGBM library designed for ranking tasks, optimizing ranking-specific objectives to produce highly accurate ordered lists in large-scale, complex datasets.

LGBRanker is a ranking algorithm that utilizes LambdaRank (see <https://quantco.com/book-lambdarank>) as its underlying optimization method.

“Learning-to-rank is the application of machine learning in the construction of ranking models for information retrieval systems. In learning-to-rank, training data consists of lists of items with some partial order specified between items in each list, and the ranking model’s purpose is to rank, i.e., produce a permutation of items in new, unseen lists.”

—Xindi Wang

The critical applications in finance are credit risk modeling, where LambdaRank can rank borrowers based on the likelihood of defaulting on loans; recommendation systems with LambdaRank providing banking products based upon a client’s transaction history and other behavioral data; customer ranking, where the clients are ranked upon their potential value to the financial institution, or portfolio optimization with

LambdaRank helping ranking assets based upon their expected returns and risk profiles.

As a tree-based algorithm, LGBRanker does not require normalization or standardization, but it can enhance its performance, especially when processing features on vastly different scales.

Python Example

Let us demonstrate using the library LGBMRanker model to rank stocks based on synthetic data (i.e., historical returns, volatility, and momentum). We introduce a target ranking based on a weighted combination of the features. We split the stocks into 10 groups, each containing 10 stocks. Each group is a distinct subset of the data for which rankings are generated and evaluated independently. An example of the grouping could be daily stock rankings: each day could be a group, and the model ranks stocks within each day. Alternatively, different portfolio market segments are groups, and the model ranks the assets within each segment.

```
import numpy as np
from lightgbm import LGBMRanker
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ndcg_score

# Step 1: Generate Synthetic Stock Data
np.random.seed(42)
n_stocks = 100
n_groups = 10

# Features: historical returns, volatility, and momentum
historical_returns = np.random.rand(n_stocks, 1)
volatility = np.random.rand(n_stocks, 1)
momentum = np.random.rand(n_stocks, 1)

# Create a feature matrix
X = np.hstack((historical_returns, volatility, momentum))

# Generate synthetic target ranking based on a combination of features
true_rank = historical_returns * 0.5 + volatility * 0.3 + momentum * 0.2
y = np.argsort(np.argsort(true_rank.flatten())) # Rank stocks

# Ensure the labels are within the expected range for LightGBM
y = np.digitize(y, bins=np.linspace(0, n_stocks, 32)) - 1

# Assume each group consists of n_stocks // n_groups stocks for ranking
groups = np.repeat(np.arange(n_groups), n_stocks // n_groups)

# Step 2: Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test, group_train, group_test = train_test_split(
    X, y, groups, test_size=0.2, random_state=42, stratify=groups)

# Define group sizes
train_group_sizes = np.bincount(group_train)
test_group_sizes = np.bincount(group_test)

# Remove zero entries in group sizes
```

```

train_group_sizes = train_group_sizes[train_group_sizes != 0]
test_group_sizes = test_group_sizes[test_group_sizes != 0]

# Step 3: Initialize and Train LGBMRanker
model = LGBMRanker(
    objective="lambdarank",
    metric="ndc",
    boosting_type="gbt",
    learning_rate=0.05,
    num_leaves=31,
    ndcg_eval_at=[1, 3, 5]
)

# Fit the model
model.fit(X_train, y_train, group=train_group_sizes, eval_set=[(X_test, y_test)], eval_group=[test_group_sizes])

# Step 4: Predict and Evaluate
y_pred = model.predict(X_test)

# Filter out groups with fewer than 2 documents
unique_groups = np.unique(group_test)
valid_groups = [g for g in unique_groups if np.sum(group_test == g) > 1]

# Calculate NDCG score for each valid group and average them
ndcg_scores = [ndcg_score([y_test[group_test == g]], [y_pred[group_test == g]], k=5) for g in valid_groups]
mean_ndcg = np.mean(ndcg_scores)
print(f'Mean NDCG Score @5: {mean_ndcg}')

# Visualization of results

# Scatter Plot of True vs. Predicted Rankings for Multiple Groups
plt.figure(figsize=(12, 8))
for group in valid_groups:
    plt.scatter(y_test[group_test == group], y_pred[group_test == group], alpha=0.6, label=f'Group {group}')

plt.xlabel('True Rankings')
plt.ylabel('Predicted Rankings')
plt.title('True vs. Predicted Rankings for Multiple Groups')
plt.legend()
plt.show()

```

With the output

```
Mean NDCG Score @5: 0.9859718699852197
```

Normalized discounted cumulative gain (NDCG) is a benchmark performance metric for evaluating the quality of a ranking. The NDCG score ranges from 0 to 1, where 1 indicates a perfect ranking. The @5 in the name of the score refers to evaluating the top five items in the ranked list.

[Figure 5.20](#) communicates a similar story to the NDCG Score @5—most points are close to the diagonal line where the model's predicted rankings match the accurate rankings, indicating the model's performance is very good.

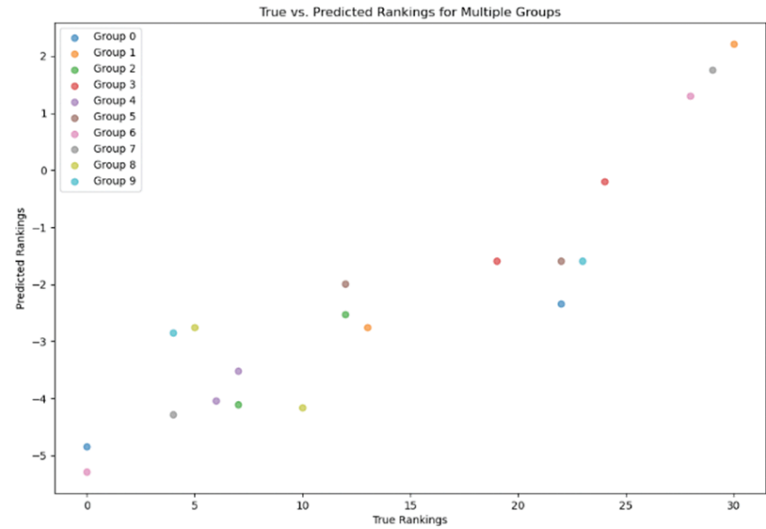


Figure 5.20 Example of true versus predicted ranking for multiple groups.

Clustering

OPTICS Clustering

Description	A density-based clustering algorithm that identifies clusters of varying densities in noisy data with an unknown number of clusters.
Key Use Cases in Finance	Detecting anomalies in transaction data, segmenting clients based on transaction patterns, and identifying market regimes.
Feature Normalization/Standardization Required	Recommended.
Python Libraries Setup	<code>pip install numpy matplotlib scikit-learn</code>

Clustering is a technique used to identify insights and patterns in complex datasets. OPTICS (**O**rding **P**oints **T**o **I**dentify the **C**lustering **S**tructure) is one of the most frequently used algorithms for noisy data with an unknown number of clusters. Unlike traditional clustering methods like k-means, which require specifying the number of clusters beforehand, OPTICS processes data points based on their density reachability and core distance, allowing it to discover the underlying clustering structure without needing a predefined number of clusters.

Key applications in finance include detecting anomalies in transaction data, segmenting clients based on behavioral patterns, and identifying different market regimes.

For optimal performance, feature normalization is recommended to ensure that all features contribute equally to the clustering process.

Python Example

Let's illustrate how to use the OPTICS algorithm in Python to generate synthetic data using the `sklearn.datasets.make_blobs` method.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import OPTICS
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Generate synthetic data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

# Standardize features (feature normalization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply OPTICS clustering
optics = OPTICS(min_samples=10, xi=0.05, min_cluster_size=0.1)
optics.fit(X_scaled)

# Extract labels and core sample indices
labels = optics.labels_
core_samples = np.zeros_like(labels, dtype=bool)

# Print numerical performance metrics
# Filter out noise points (-1) for metrics calculation
mask = labels != -1
print("Silhouette Score:", silhouette_score(X_scaled[mask], labels[mask]))
print("Davies-Bouldin Index:", davies_bouldin_score(X_scaled[mask], labels[mask]))

# Plot the clustering results
plt.figure(figsize=(10, 6))
unique_labels = set(labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black used for noise.
        col = [0, 0, 0, 1]

    class_member_mask = (labels == k)
    xy = X_scaled[class_member_mask & core_samples]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col), markeredgecolor='k', markersize=8)

    xy = X_scaled[class_member_mask & ~core_samples]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col), markeredgecolor='k', markersize=8)

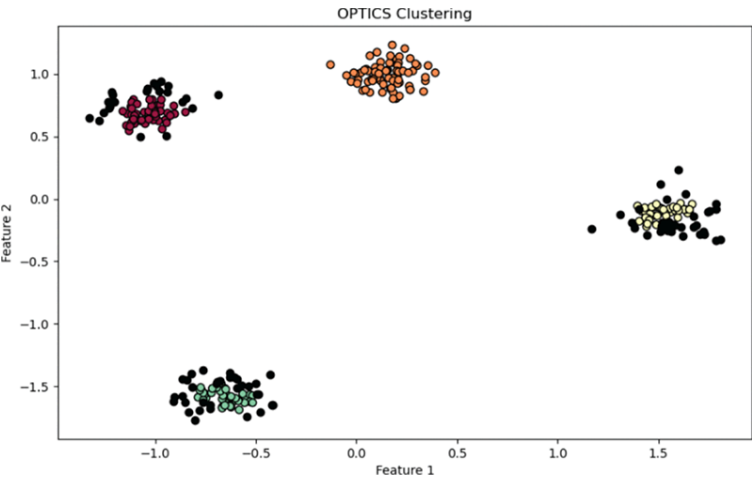
plt.title('OPTICS Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

With the output

```
Silhouette Score: 0.9007128264042205
Davies-Bouldin Index: 0.13502085379361267
```

Silhouette score (the higher, the better clustering) measures how similar each point is to its cluster compared to others. Davies-Bouldin index (the lower, the better clustering) measures the average similarity ratio of each cluster compared to its most similar cluster.

The black color is used in [Figure 5.21](#) to indicate noise.



[Figure 5.21](#) Example of OPTICS clustering.

Language Models

OpenAI Language Model

Description	Large language models (LLMs) are models capable of understanding and generating human-like text, making them valuable for processing and analyzing large volumes of financial data.
Key Use Cases in Finance	Market analysis and research, regulatory compliance, and reporting.
Feature Normalization/Standardization Required	Not required.
Python Libraries Setup	<code>pip install openai json</code>

Large language models (LLMs) like OpenAI's GPT series are developed using vast textual datasets. They are engineered to process and produce text that resembles human writing and performing tasks, such as summariz-

ing documents, translating languages, responding to queries, composing text, and evaluating sentiment.

LLMs are advanced AI systems designed to understand and generate human-like text based on vast amounts of data. Their key advantages are their ability to analyze and interpret complex language patterns, develop coherent and contextually relevant text, and provide insights from unstructured data. However, LLMs can be computationally intensive and require substantial data for training to achieve high performance.

In finance, LLMs are utilized for sentiment analysis, automated report generation, predictive analytics, and risk management.

Normalization is generally unnecessary for textual data used in LLMs, but normalizing these features can enhance the model's performance and consistency when numerical features accompany the text.

Python Example

```
import openai
import json

# Set up your OpenAI API key
openai.api_key = 'YOUR_API_KEY_HERE'

def analyze_sentiment(text):
    response = openai.chat.completions.create(
        model="gpt-4",
        messages=[
            {"role": "user", "content": f "Analyze the sentiment of the following text and provide"}
        ]
    )
    return json.loads(response.choices[0].message.content)['sentiment_score']

def test_analyze_sentiment():
    test_sentences = [
        "'I absolutely love this new phone!It's fantastic and exceeds all my expectations'",
        "'This is the worst experience I have ever had with a company. Totally unacceptable'",
        "'Meh, the movie was just okay. Not too good, not too bad'",
        "'I am thrilled with my new job. The team is great and the work is fulfilling'",
        "'The food was terrible, and the service was even worse. I am never coming back here'",
        "'Wow, what a beautiful day! The sun is shining and everything feels perfect'",
        "'This product is a complete waste of money. It broke after one use'",
        "'I am feeling pretty neutral about this situation, neither happy nor sad'",
        "'The concert last night was amazing! Best performance I've ever seen'",
        "'Ugh, what a horrible traffic jam. Made me late for work and ruined my day'"
    ]

    for i, sentence in enumerate(test_sentences):
        try:
            sentiment_score = analyze_sentiment(sentence)
            print(f"Test {i+1}: Sentence: \"{sentence}\"")
            print(f"Sentiment Score: {sentiment_score}\n")
        except Exception as e:
            print(f"Test {i+1}: Sentence: \"{sentence}\"")
            print(f"Error: {e}\n")
```



```
# Run the test method
test_analyze_sentiment()
```

With the outputs

```
Test 1: Sentence: "I absolutely love this new phone! It's fantastic and exceeds all my expectation
Sentiment Score: 9.5

Test 2: Sentence: "This is the worst experience I have ever had with a company. Totally unacceptab
Sentiment Score: 0

Test 3: Sentence: "Meh, the movie was just okay. Not too good, not too bad."
Sentiment Score: 5

Test 4: Sentence: "I am thrilled with my new job. The team is great and the work is fulfilling!"
Sentiment Score: 9.5

Test 5: Sentence: "The food was terrible, and the service was even worse. I am never coming back h
Sentiment Score: 1

Test 6: Sentence: "Wow, what a beautiful day! The sun is shining and everything feels perfect."
Sentiment Score: 9

Test 7: Sentence: "This product is a complete waste of money. It broke after one use."
Sentiment Score: 1.5

Test 8: Sentence: "I am feeling pretty neutral about this situation, neither happy nor sad."
Sentiment Score: 5

Test 9: Sentence: "The concert last night was amazing! Best performance I've ever seen."
Sentiment Score: 9.5

Test 10: Sentence: "Ugh, what a horrible traffic jam. Made me late for work and ruined my day."
Sentiment Score: 2
```

Amazon Chronos Model

Description	The Chronos model is a pretrained time series forecasting model.
Key Use Cases in Finance	Asset prices/trends prediction.
Feature Normalization/Standardization Required	Required.
Python Libraries Setup	<pre>pip install git+https://github.com/amazon-science/chronos-forecasting.git pip install torch pandas numpy matplotlib sklearn</pre>

Chronos (see qnt.co/book-amazon-chronos-tiny and qnt.co/book-amazon-chronos-large) is a collection of ready-to-use time series forecasting models based on language model architectures. A time series is converted into a sequence of tokens by scaling and quantizing, and a language model is trained on these tokens using the cross-entropy loss. Chronos models have been trained on a large corpus of public time series data and artificial data created using Gaussian processes.

In finance, the Chronos model is suitable for time series forecasting to predict future asset prices, trends, or market movements, aiding in portfolio optimization and risk management.

Normalization or standardization is essential to ensure faster convergence and better performance.

For production use, the pretrained models, such as the Chronos models, are fine-tuned to the task-specific dataset to obtain enhanced performance. Later examples in the book illustrate this process.

Python Example

Let us create a synthetic time series and apply the Chronos model to forecast 30 days.

Notice the following line:

```
forecast_values = np.median(forecast, axis=0)
```

The predicted output is an array with rows of several predicted samples and columns of the predicted values for each of the 30 forecasted days. This line produces a single time series of medians of the expected values.

Instead, we could consider generating the 10%, 50%, and 90% quantiles, providing us with the range of likely values:

```
low, median, high = np.quantile(forecast, [0.1, 0.5, 0.9], axis=0)
```

The complete source code follows:

```
import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from chronos import ChronosPipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Load the pre-trained Chronos model
model = ChronosPipeline.from_pretrained(
    "amazon/chronos-t5-tiny",
    device_map="cuda" if torch.cuda.is_available() else "cpu",
    torch_dtype=torch.bfloat16,
)
```

```

# Generate a representative time series (e.g., sine wave with noise)
np.random.seed(0)
dates = pd.date_range(start='2022-01-01', periods=200, freq='D')
series = np.sin(np.linspace(0, 20, 200)) + np.random.normal(0, 0.5, 200)
data = {'date': dates, 'close': series}
df = pd.DataFrame(data)

# Normalize the data
scaler = StandardScaler()
normalized_data = scaler.fit_transform(df[['close']])

# Convert normalized data to a PyTorch tensor
tensor_data = torch.tensor(normalized_data)

# Forecast future prices for the next 30 days
prediction_length = 30
forecast = model.predict(tensor_data, prediction_length)

# Convert forecast back to original scale
forecast = scaler.inverse_transform(forecast[0].numpy())

# Combine original and forecasted data for visualization
forecast_dates = pd.date_range(start=dates[-1] + pd.Timedelta(days=1), periods=prediction_length,

forecast_values = np.median(forecast, axis=0)

forecast_df = pd.DataFrame({'date': forecast_dates, 'forecast': forecast_values})
combined_df = pd.concat([df[['date', 'close']], forecast_df], axis=0)

# Plot the original and forecasted data
plt.figure(figsize=(12, 6))
plt.plot(df['date'], df['close'], label='Original')
plt.plot(forecast_df['date'], forecast_df['forecast'], label='Forecast', linestyle='--')
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.title('Original vs Forecasted Closing Prices')
plt.legend()
plt.show()

# Calculate and print performance metrics
# For simplicity, compare last 30 days of original data with forecast
actual = df['close'].values[-prediction_length:]
predicted = forecast.flatten()[:len(actual)]

mse = mean_squared_error(actual, predicted)
mae = mean_absolute_error(actual, predicted)

print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")

```

With the output ([Figure 5.22](#))

```

Mean Squared Error (MSE): 6.4039
Mean Absolute Error (MAE): 2.4492

```

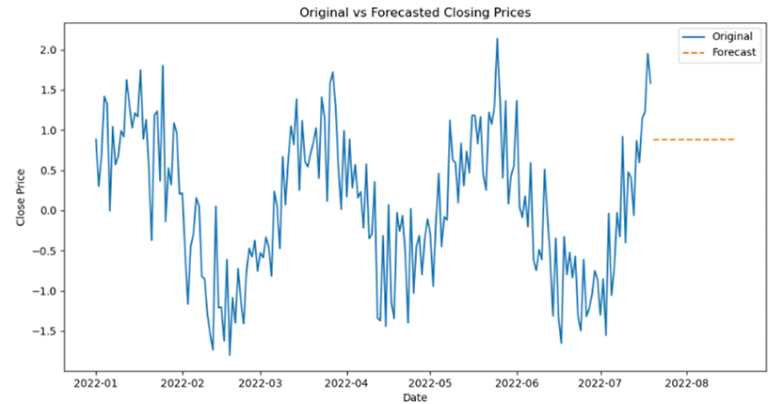


Figure 5.22 Example of Amazon Chronos prediction.

FinBERT Model

Description	The FinBERT model is a pretrained financial sentiment analysis model designed to classify the sentiment of financial texts using the BERT architecture. It has been optimized to understand the language used in the financial domain.
Key Use Cases in Finance	Sentiment analysis of financial texts, market prediction, risk management, portfolio management, customer feedback analysis, and assessing the impact of events to inform investment decisions and understand market and investor reactions.
Feature Normalization/Standardization Required	Required.
Python Libraries Setup	<code>pip install tensorflow transformers tf-keras</code>

FinBERT (qnt.co/book-finbert) is a pretrained natural language processing (NLP) model designed to analyze the sentiment of financial texts. It is developed by further training the BERT language model on a large corpus of financial data, fine-tuning it specifically for financial sentiment classification.

The FinBERT model is used in finance for sentiment analysis of financial texts, such as news, reports, and social media, to understand market sentiment and investor mood. It aids in predicting market movements, managing risks, and informing portfolio management strategies by correlat-

ing sentiment trends with stock price movements. Finally, FinBERT is used to analyze customer feedback for improving financial products and services and to assess the impact of specific events, such as earnings reports or mergers, on market sentiment.

Normalization or standardization is essential to ensure faster convergence and better performance.

For production use, the pretrained models, such as the FinBERT models, are fine-tuned to the task-specific dataset to obtain enhanced performance. A later example in the book illustrates this process.

Python Example

Let us illustrate how to use the FinBERT model's sentiment analysis on a set of synthetic financial texts. The texts are first preprocessed to generate tokenized inputs; an attention mask is created to ensure the model focuses on relevant parts. Sentiment predictions are then obtained, and the results are printed with corresponding sentiment labels ("negative," "positive").

```
import random
import numpy as np
import tensorflow as tf
from transformers import BertTokenizer, TFBertForSequenceClassification, BertConfig

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

# Load the tokenizer and the model
model_path = "ProsusAI/finbert"
tokenizer = BertTokenizer.from_pretrained(model_path)
model = TFBertForSequenceClassification.from_pretrained(
    model_path,
    config=BertConfig.from_pretrained(model_path, num_labels=2)
)

# Create longer synthetic financial data
texts = [
    "The purchase price will be paid in cash and stock upon the closure of the transaction, schedu",
    "With this, the company will exit the contract manufacturing service segment and enter the mor",
    "Commission income fell to SGD 2.1 mn from SGD 5.1 mn in the corresponding period in 2021",
    "Finnish media group Talentum has issued a profit warning followed by no dividend payment for",
    "The loss for the second quarter of 2024 was EUR 0.1 mn smaller than the loss of the first qua"
]

# Preprocess the texts
inputs = tokenizer(
    texts,
    return_tensors='tf',
    padding=True,
    truncation=True,
    max_length=512,
    add_special_tokens=True
)

# Ensure model focuses on relevant parts using attention mask
```

```

inputs['attention_mask'] = tf.cast(inputs['attention_mask'], dtype=tf.float32)

# Get sentiment predictions
outputs = model(**inputs)
predictions = tf.nn.softmax(outputs.logits, axis=-1).numpy()

# Define the sentiment labels
labels = ['negative', 'positive']

# Print the results
for i, text in enumerate(texts):
    sentiment = labels[tf.argmax(predictions[i]).numpy()]
    confidence = tf.reduce_max(predictions[i]).numpy()
    print(f"Text: {text}\nSentiment: {sentiment} (Confidence: {confidence:.2f}); Predictions: {pre

```

With the output

```

Text: The purchase price will be paid in cash and stock upon the closure of the transaction, sched
Sentiment: negative (Confidence: 0.59); Predictions: [0.5932849 0.4067151]

Text: With this, the company will exit the contract manufacturing service segment and enter the mo
Sentiment: negative (Confidence: 0.53); Predictions: [0.52938175 0.47061822]

Text: Commission income fell to SGD 2.1 mn from SGD 5.1 mn in the corresponding period in 2021
Sentiment: negative (Confidence: 0.52); Predictions: [0.52257305 0.47742692]

Text: Finnish media group Talentum has issued a profit warning followed by no dividend payment for
Sentiment: negative (Confidence: 0.53); Predictions: [0.5345768 0.46542326]

Text: The loss for the second quarter of 2024 was EUR 0.1 mn smaller than the loss of the first qu
Sentiment: positive (Confidence: 0.59); Predictions: [0.4091295 0.59087044]

```